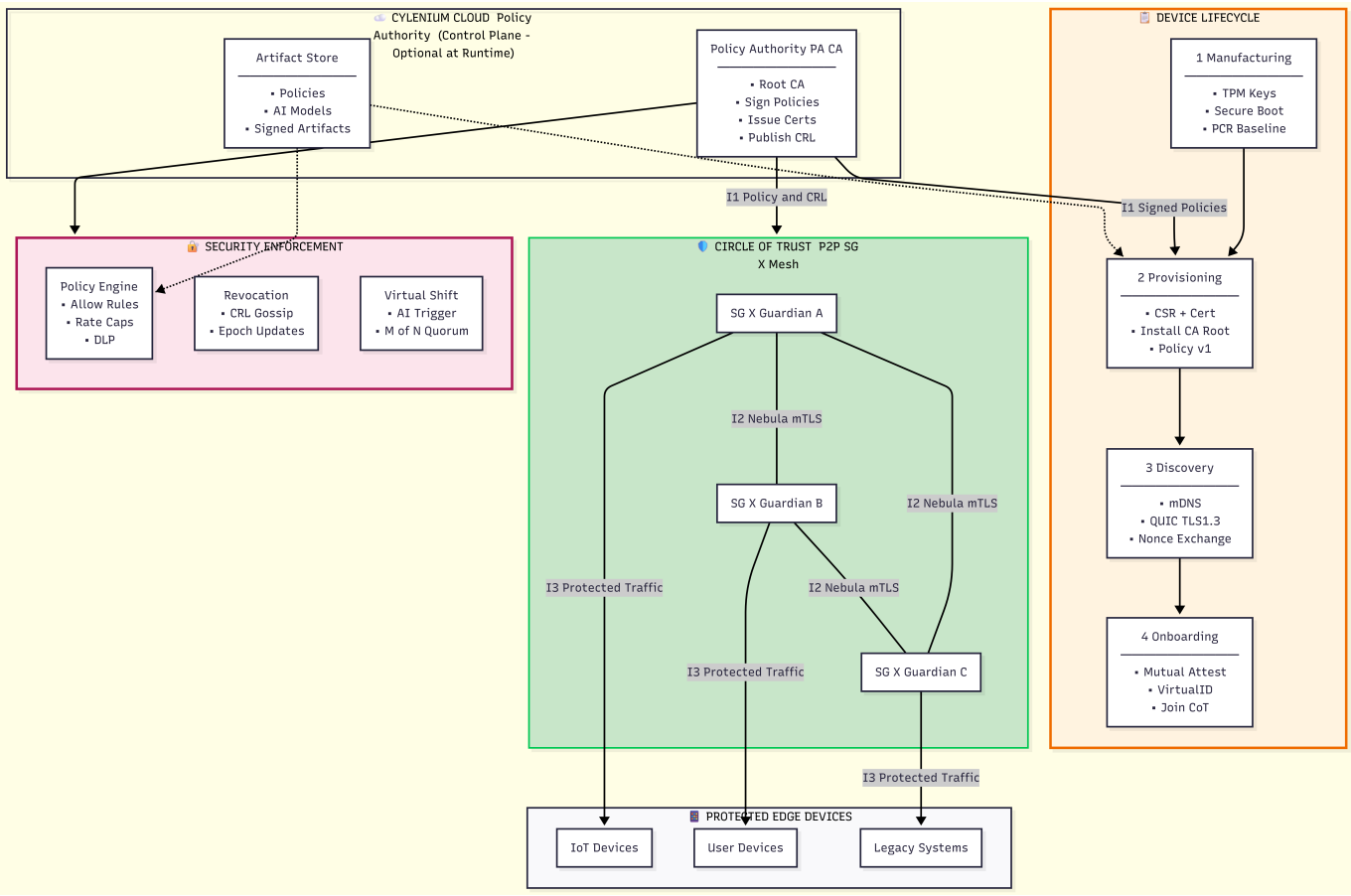
SECTION 0

0. Overall Scenario Metadata

0.1 What is the trust model for this network?

Answer: SG-X uses a hybrid zero-trust architecture: identity issuance is centralized via the PA-CA root, while runtime authentication, attestation, authorization, and policy enforcement are fully distributed across all SG-X nodes. Devices do not trust anything based on network location; instead, they validate peers using certificates, attestation evidence, VirtualID, and policy alignment.

High Level Architecture Diagram:



0.2 Is there a single root of trust (manufacturer CA, enterprise CA, local controller), or multiple?

Answer: The primary root of trust is the fleet-level PA-CA, which issues device certificates and signs policies. Additionally, SG-X allows Circle-level CAs (owned by an administrator or Guardian) for domain-local trust delegation. Devices validate certificate chains against these authorized CA roots.

0.3 Is trust centralized (controller or cloud) or distributed (peer-to-peer circle of trust)?

Answer: Certificate issuance, policy signing, and revocation authority are centralized via PA-CA. All runtime trust decisions — authentication, attestation, authorization, and CRL enforcement — are distributed across nodes. SG-X operates even when cloud connectivity is unavailable.

Centralized (PA-CA)	Distributed (Each Node)
- Cert issuance	- Cert validation
- Policy signatures	- Attestation checks
- CRL signing	- Authorization decisions
	- CRL enforcement

0.4 What are the primary security goals?

Answer: SG-X security objectives include:

Confidentiality: QUIC/TLS1.3 + ECH, Nebula encrypted overlay, PQC-hybrid key exchange.

Integrity & Authenticity: Certificate-based authentication, attestation (PCRs + policy_digest), signed policies.

Anti-cloning: Non-exportable TPM/TEE keys, secure boot, attestation-bound VirtualID.

Availability: Local enforcement with cloud optionality; Nebula multi-path connectivity.

Revocation & Recovery: Distributed epoch-based CRL gossip ensuring fail-closed behavior.

0.5 What are the assumed attacker capabilities?

Answer: The attacker model assumes a high-capability adversary who may:

Control or manipulate Wi-Fi/LAN traffic

Perform MITM or replay attempts

Physically access devices (JTAG, flash extraction)

Temporarily steal and return devices

Introduce rogue/fake devices into the network

Attempt long-term cryptanalysis (“harvest now, decrypt later”)

SG-X mitigates these threats with hardware-secured keys, attestation, VirtualID, PQC-hybrid KEX, CRL gossip, secure boot, and fail-closed validation logic.

0.6 Can the attacker control the Wi-Fi network?

Answer: Yes. The architecture assumes adversaries may control the Wi-Fi/LAN. Discovery is therefore unauthenticated, and trust is established only after certificate validation, QUIC/TLS1.3 handshake with ECH, and mutual attestation. Even with full control of the local network, attackers cannot impersonate SG-X nodes.

0.7 Can they physically access a device (connect JTAG, read flash)?

Answer: Physical access is assumed possible. SG-X protects keys using TPM/TEE secure elements, hardware-rooted encryption for sensitive material, and secure boot. Attestation ensures firmware or bootloader tampering is detected, causing peers to reject the device.

0.8 Can they temporarily steal a device and return it?

Answer: Yes. SG-X includes mitigations for temporary device theft:

PCR and policy_digest mismatches will fail attestation

Administrator can activate Lost Mode

Revocation via CRL gossip blocks the device returning to the Circle

VirtualID rotation and session expiry prevent replay of prior authenticated states

```
Stolen → Mark Lost → CRL Update → Gossip → Device Blocked
```

SECTION 1

1. Initial Single Device: NodeA (Cervais SG-X)

1.1 Manufacturing & Provisioning

Answer: Each SG-X device is provisioned with its identity and attestation foundations during manufacturing through a secure, hardware-backed process:

- Device-Unique Identity Keypair: Generated inside the TPM/TEE, ensuring the private key is non-exportable. Used later for certificate-based authentication and mutual TLS.
- Device-Unique Attestation Keypair: A separate TPM-backed key used exclusively for attestation. It signs PCR (Platform Configuration Register) values and the active policy_digest to prove software integrity and configuration state.
- No Shared or Batch Keys: SG-X prohibits shared symmetric keys or manufacturer debug keys. All secrets are device-unique to avoid fleet-wide compromise.
- Key Storage:
    - Identity & attestation private keys → TPM/TEE
    - Certificates & metadata → secure flash
    - Sensitive configuration → encrypted with hardware root

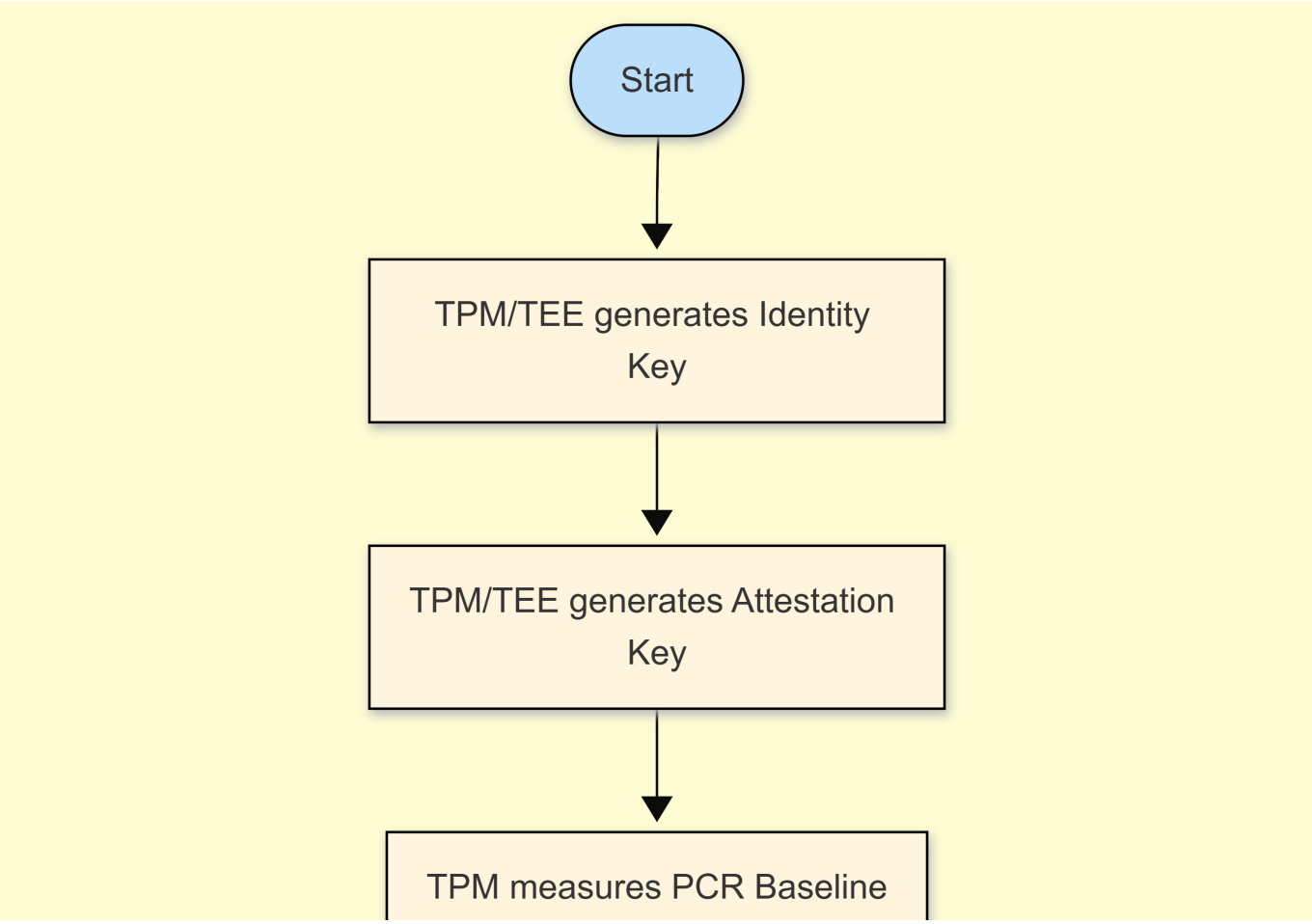
- Certificates & Trust Anchors Installed: During provisioning, the device sends a CSR (Certificate Signing Request) containing its public key and baseline PCRs to the PA-CA.

```
PA-CA returns:

- device certificate (device.crt)

- PA-CA root certificate

- optional intermediate CAs
```

- Initial Policy Installation (Policy v1): The first policy is installed using an atomic A/B mechanism:
  - Policy staged into inactive slot
  - Signature & structure verified
  - policy_digest computed
  - Atomic activate
  - Device now reports policy_digest=v1 in attestation

This ensures the device begins life in a verified, policy-bound, and attestation-ready state.

Diagram B-0 — Manufacturing



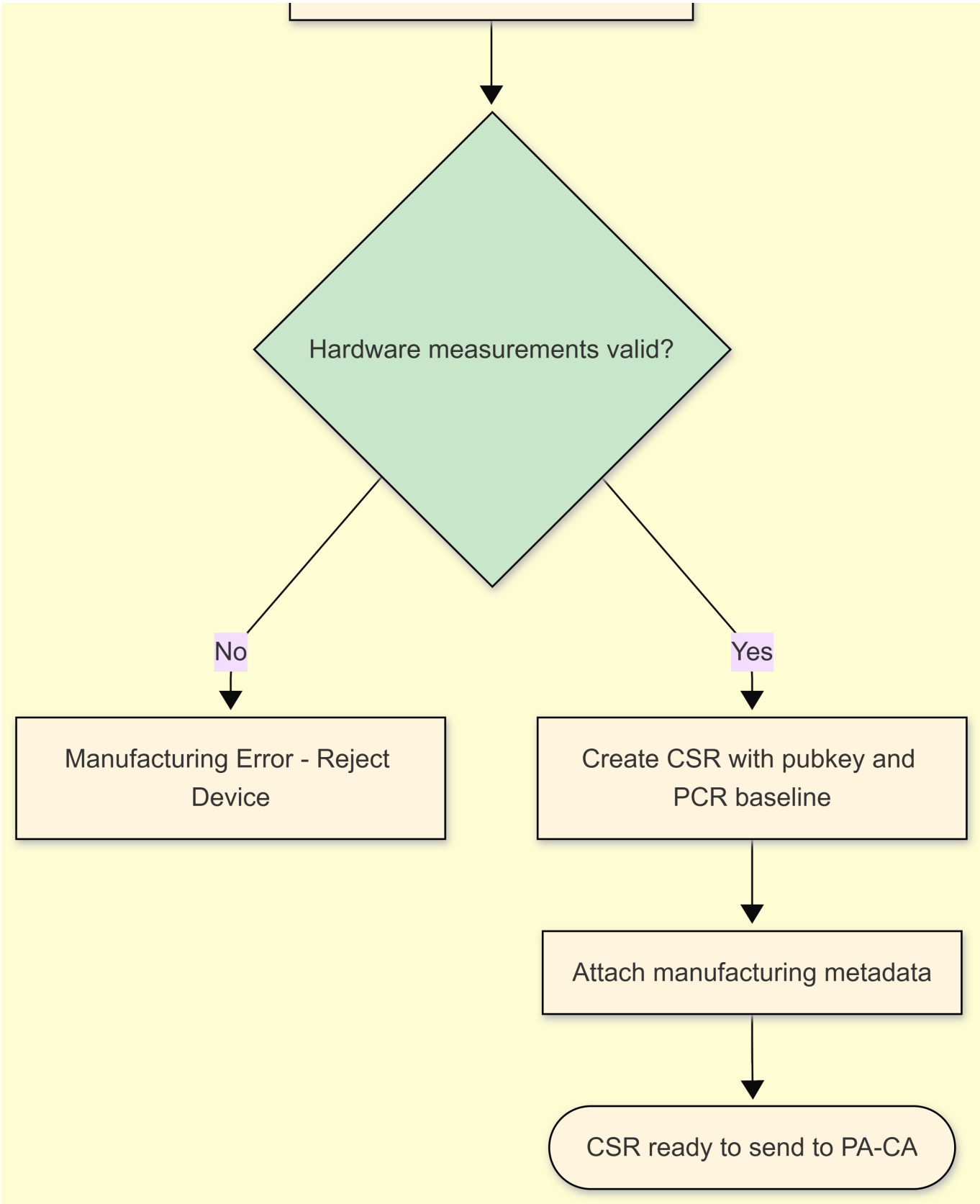
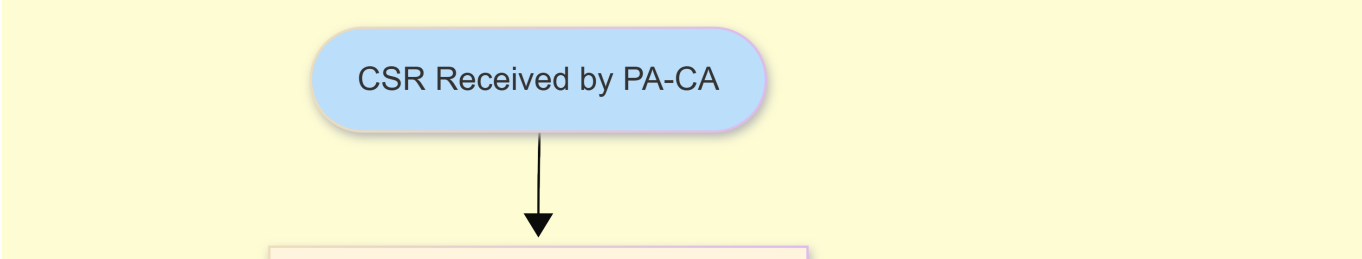
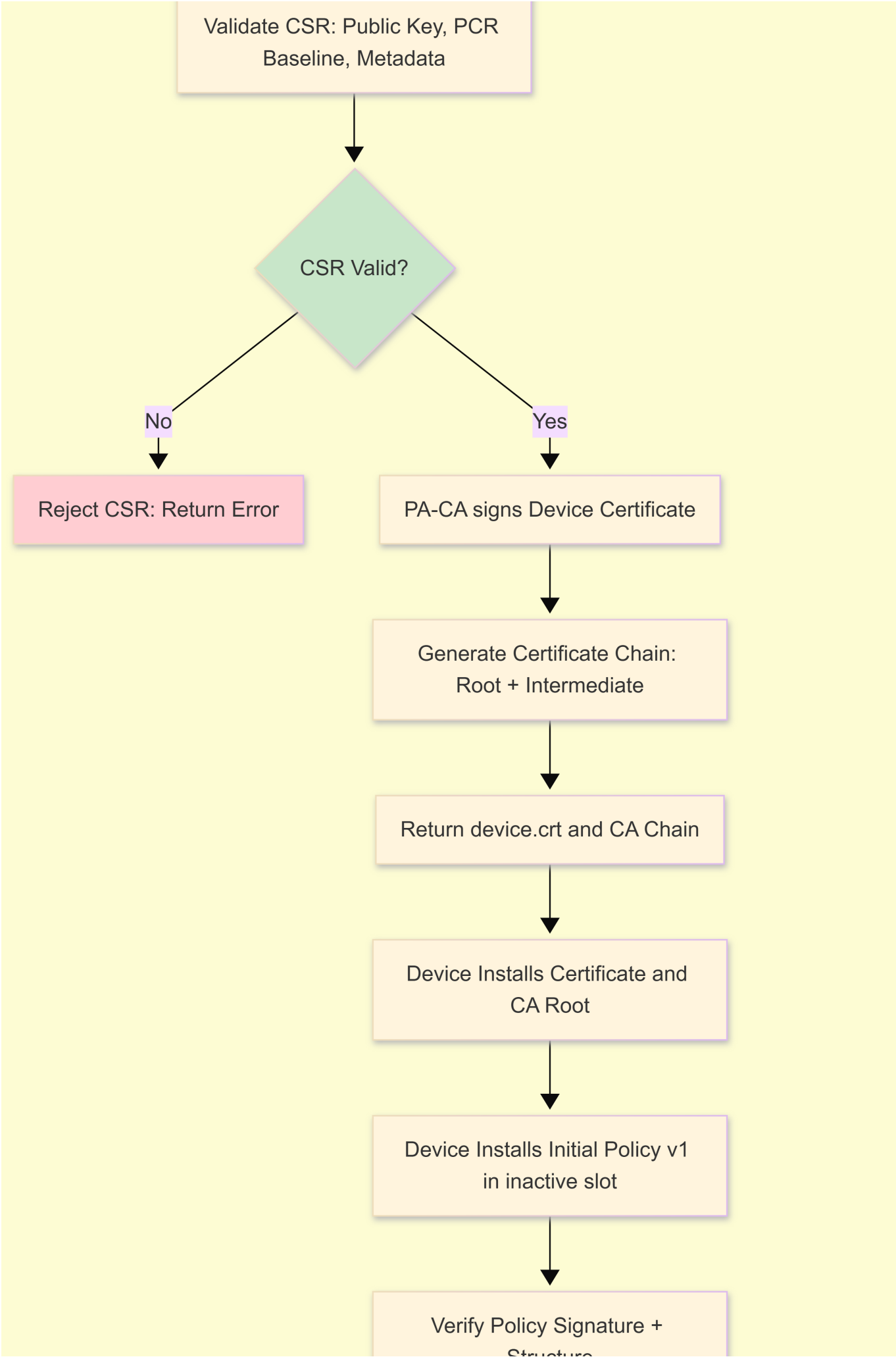
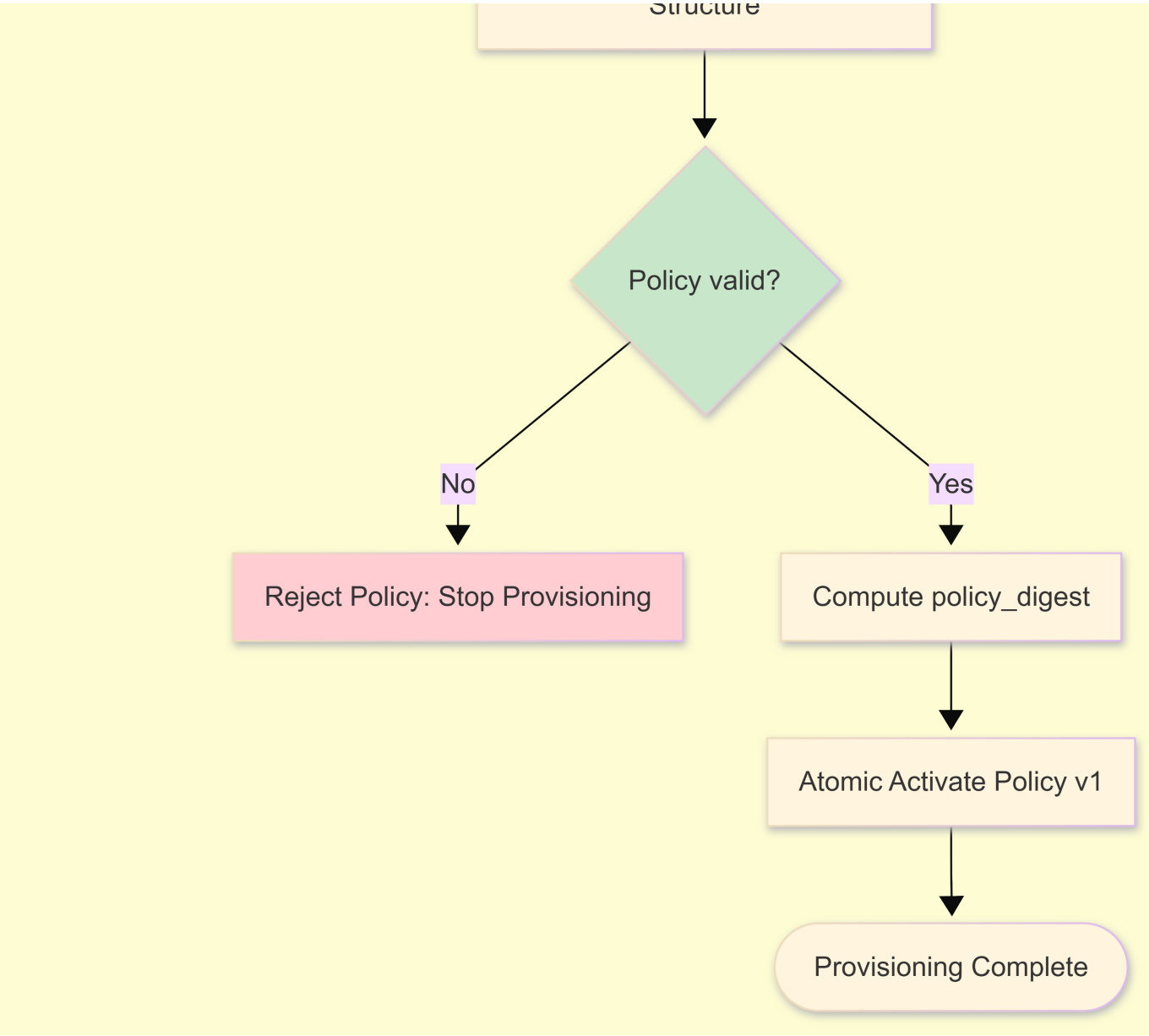


Diagram B-1 — Provisioning




Validate CSR: Public Key, PCR Baseline, Metadata
CSR Valid?
No
Yes
Reject CSR: Return Error
PA-CA signs Device Certificate
Generate Certificate Chain: Root + Intermediate
Return device.crt and CA Chain
Device Installs Certificate and CA Root
Device Installs Initial Policy v1 in inactive slot
Verify Policy Signature +



1.2 Identity & Attestation

Answer: NodeA's long-term identity is its PA-CA–signed device certificate, which binds a hardware-protected public key to the device identity. This certificate, combined with secure boot measurements, forms the anchor for authentication.

To prove the device is genuine, NodeA performs TPM-backed attestation, producing:

```
Attest(
   PCRs,
   policy_digest,
   Nonce_I,
   Nonce_R,
   sig_attestation_key
)
```

Peers validate:

PCR values → firmware and boot chain integrity

policy_digest → confirms device is enforcing correct signed policy

Signature → valid attestation key

Certificate chain → trusted PA-CA hierarchy

Only after all validations succeed is the device accepted into the Circle-of-Trust.

Mini Diagram — Identity Validation

```
NodeA → NodeB
------------------------
• device.crt
• Attest(PCRs, policy_digest, nonces, sig_AK)

NodeB verifies:
   - CA chain (PA-CA)
   - Attestation signature
   - PCR integrity
   - policy_digest correctness

If all valid → trust established.
```

1.3 Key Usage & Lifecycle

Answer: SG-X devices maintain a strict separation of cryptographic responsibilities:

- Identity Key: Used exclusively for authentication in mTLS and certificate validation. Stored in TPM.
- Attestation Key: Used to sign PCR measurements and policy_digest during attestation. Ensures hardware + policy integrity.
- Session Keys: Derived from QUIC/TLS1.3 handshake using ECDHE or an X25519+Kyber hybrid exchange. Forward secrecy ensures past traffic remains secure even if long-term keys are compromised.
- Firmware Verification Keys: Stored in secure ROM or protected flash. Used to verify signed firmware images during secure boot.
- Key Rotation:
    - TLS session keys rotate per session
    - Certificates renewable in the field
    - policy_digest updates trigger a VirtualID update
    - Revoked keys propagated by CRL gossip
- Factory Reset Behavior: Wipes configuration, policies, CRLs, and cached state. Identity keys may persist or be reprovisioned depending on enterprise policy. Attestation re-validates baseline on reboot.

Mini Diagram — Key Roles

```
Identity Key     → Authentication (mTLS)
Attestation Key  → Attestation(PCRs + policy_digest)
Session Keys     → Encrypted runtime communication
Firmware Key     → Verify signed firmware
```

SECTION 2

2. Introduce NodeB to the Same Network

We introduce a second Cervais SG-X device (NodeB) to the network.

2.1 Discovery

What discovery mechanisms are currently supported between SG-X devices on the same network?

Answer:

SG-X supports two complementary discovery layers:

1. LAN Discovery

- Multicast DNS (mDNS / DNS-SD)
- Broadcast UDP beacons
- Used to announce presence on the local subnet
- Messages are plaintext and intentionally unauthenticated

2. Global Discovery (Any Network)

- Provided by the Nebula overlay mesh
- Supports IPv4/IPv6, Wi-Fi, Ethernet, LTE/5G, satellite, and NAT traversal
- Ensures nodes can find each other even across complex networks

Discovery only indicates “a peer might exist.”

Trust is NEVER based on discovery — only on certificates + attestation.

Are discovery messages authenticated or plaintext?

Answer:

Discovery messages are plaintext and unauthenticated, by design. Actual authentication occurs during the handshake:

- QUIC/TLS1.3 (with ECH, GREASE, padding)
- Certificate validation
- Attestation validation

Thus spoofing discovery is harmless.

Can an attacker spoof a “fake SG-X” in discovery?

Answer:

Yes, but spoofing discovery is meaningless, because the fake device will always fail:

- Certificate chain validation
- Attestation (PCR + policy_digest mismatch)
- VirtualID computation

False discovery ≠ false trust.

How does NodeB learn the existence of NodeA?

Answer:

NodeB can learn about NodeA through:

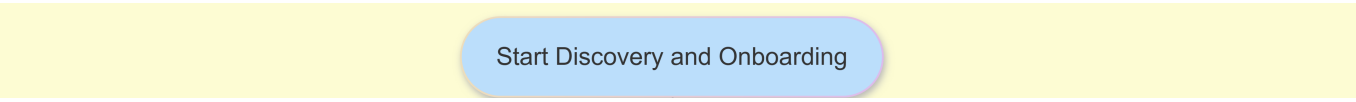
- Periodic mDNS advertisements from NodeA
- Nebula lighthouse lookups
- NodeB actively probing (“Who is out there?”)

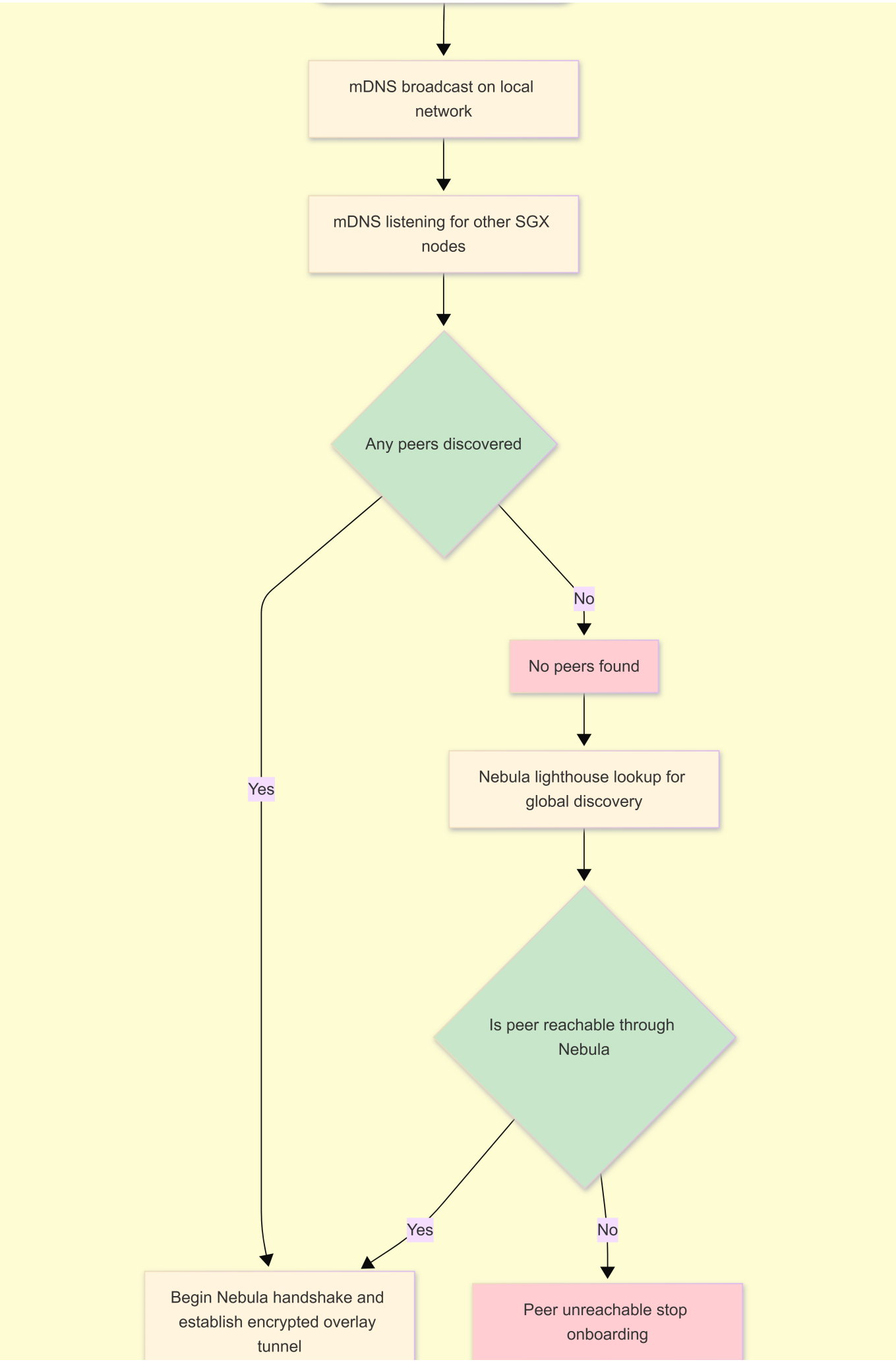
Both passive and active discovery methods are supported.

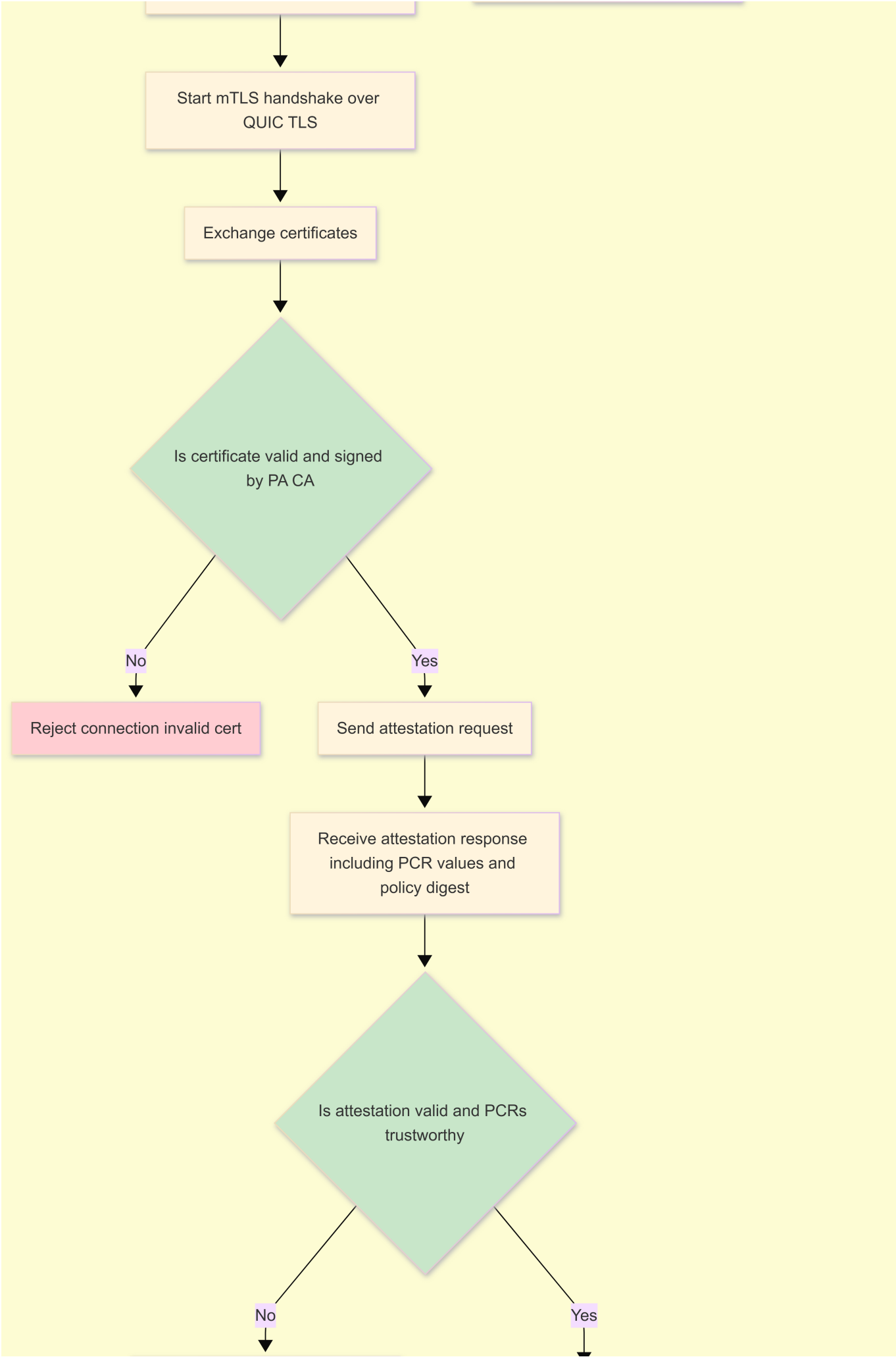
Mini Diagram — LAN Discovery vs Global Discovery

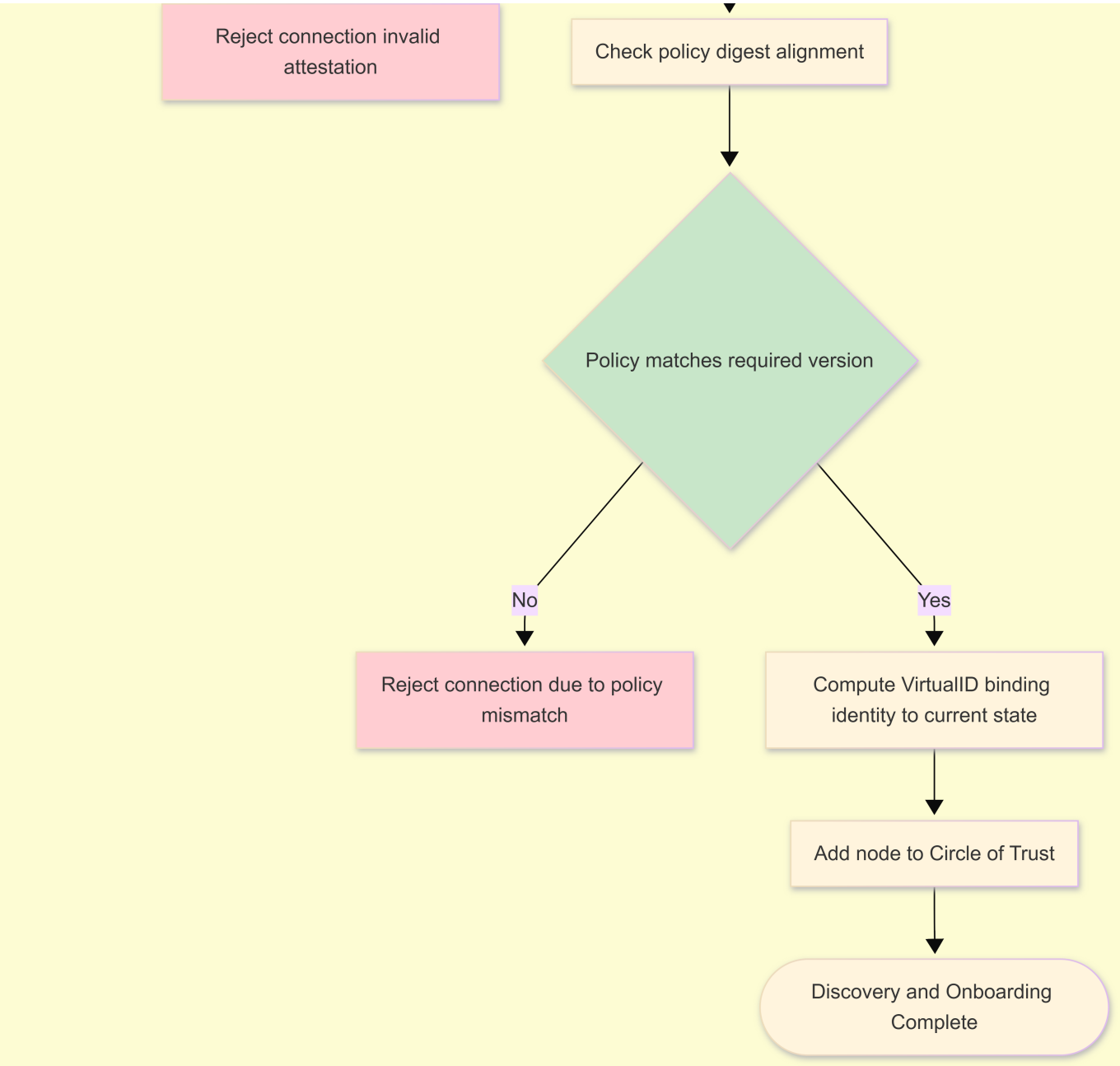
```
LAN (mDNS)                                  Nebula (Global)
──────────                                  ──────────────────────────
NodeA → "_sgx._tcp.local" beacon            NodeA → Lighthouse Register
NodeB ← discovers A                         NodeB ← Lookup via Lighthouse
(Untrusted)                                 (Secure overlay ready)
```

Diagram B-2 — Discovery & Onboarding




mDNS broadcast on local network
mDNS listening for other SGX nodes
Any peers discovered
Yes
No
No peers found
Nebula lighthouse lookup for global discovery
Is peer reachable through Nebula
Yes
No
Begin Nebula handshake and establish encrypted overlay tunnel
Peer unreachable stop onboarding


Start mTLS handshake over QUIC TLS
Exchange certificates
Is certificate valid and signed by PA CA
No
Yes
Reject connection invalid cert
Send attestation request
Receive attestation response including PCR values and policy digest
Is attestation valid and PCRs trustworthy
No
Yes



2.2 Supported Protocols (Current & Future)

Transport / Network Stack

Answer:

SG-X supports:

- Wi-Fi
- Ethernet
- Cellular (LTE/5G)
- Satellite links
- IPv4 & IPv6

- Nebula mesh overlay (global connectivity, NAT traversal)

Intermittent connectivity is fully supported.

Application-Level Protocols

Answer:

Primary communication runs over:

- gRPC over QUIC/TLS1.3
- Protobuf-based messages

Other supported capabilities:

- Secure messaging channels
- Remote command & telemetry streams
- File transfer / update channels

Security Protocols

Answer:

SG-X uses:

- QUIC/TLS1.3 with Encrypted ClientHello (ECH)
- GREASE + padding for traffic-analysis resistance
- Ephemeral ECDHE (X25519)
- Optional PQC-hybrid key exchange (X25519 + Kyber)
- Strict mutual TLS (mTLS) for all peer connections

Cipher suites are restricted to modern, secure sets only.

Roadmap (Future Protocol Support)

Answer:

Planned expansions include:

- New transports: Thread, Matter, BLE, LoRa
- New security modes: Full PQC-only handshake
- New app-level integrations: OPC-UA, Matter

- Version negotiation inside QUIC to maintain backward compatibility

---

2.3 Authorization & Authentication (NodeB → NodeA)

This section describes how NodeB becomes a trusted member communicating with NodeA.

2.3.1 Authentication

What credentials does NodeB present?

Answer:

NodeB presents:

- device.crt (PA-CA / Circle-CA signed)
- Full certificate chain
- Later: attestation evidence (PCRs + policy_digest + nonces)

Pre-shared keys or QR onboarding are not used for SG-X → SG-X trust.

How does NodeA verify NodeB's identity?

Answer:

NodeA checks:

- Certificate chain → PA-CA root
- Signature validity
- Attestation signature (AK-signed)
- PCR integrity baseline
- policy_digest = expected
- CRL status
- VirtualID correctness

NodeA maintains a trust store with:

- The PA-CA root
- Optional Circle-CA roots
- Optional enterprise intermediates

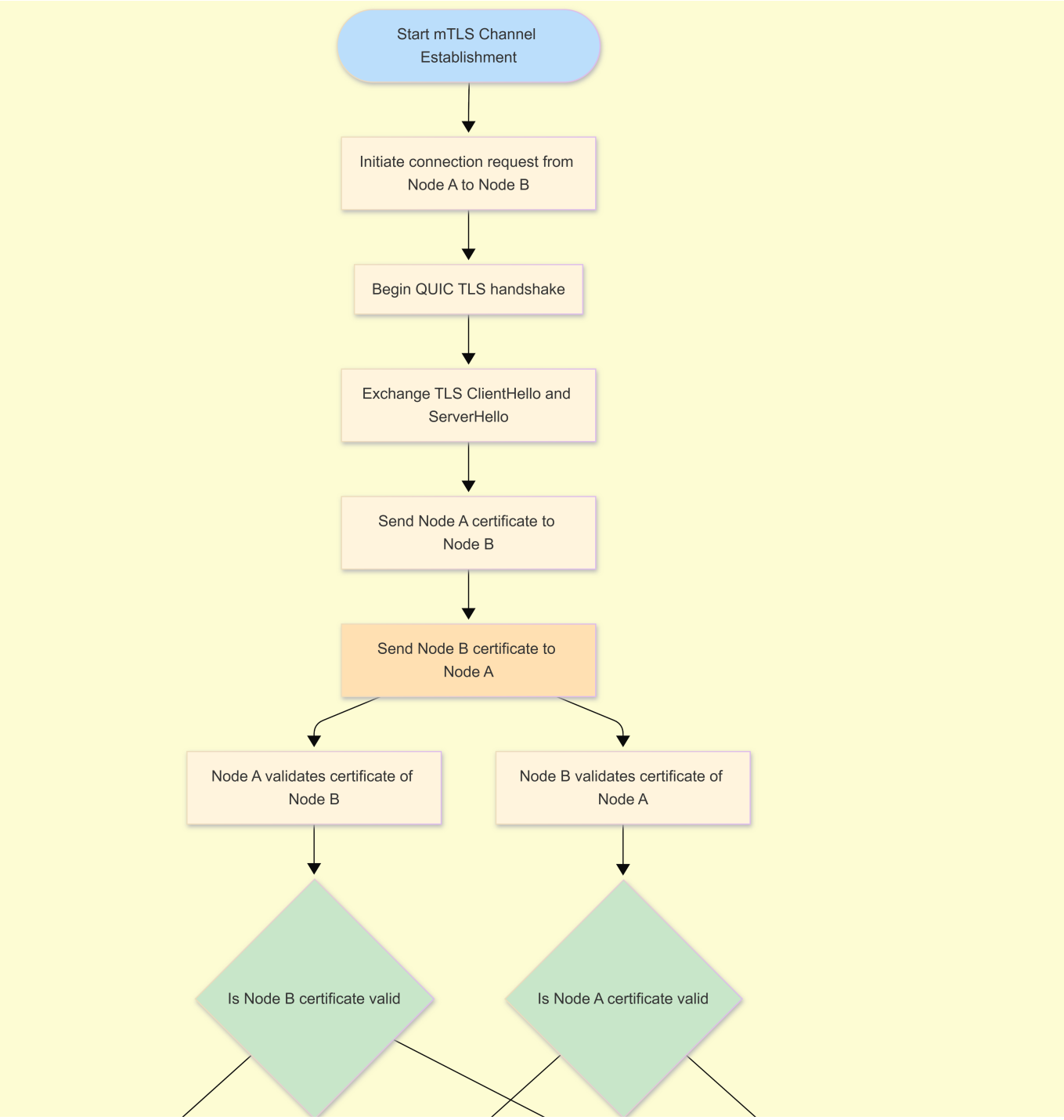
Is there mutual authentication?

Answer:

Yes. All SG-X communications require mutual TLS + mutual attestation.

- Both sides must:
- Present certificates
- Validate chains
- Provide attestation evidence
- Produce matching VirtualIDs

Diagram B-7 — mTLS Channel Establishment



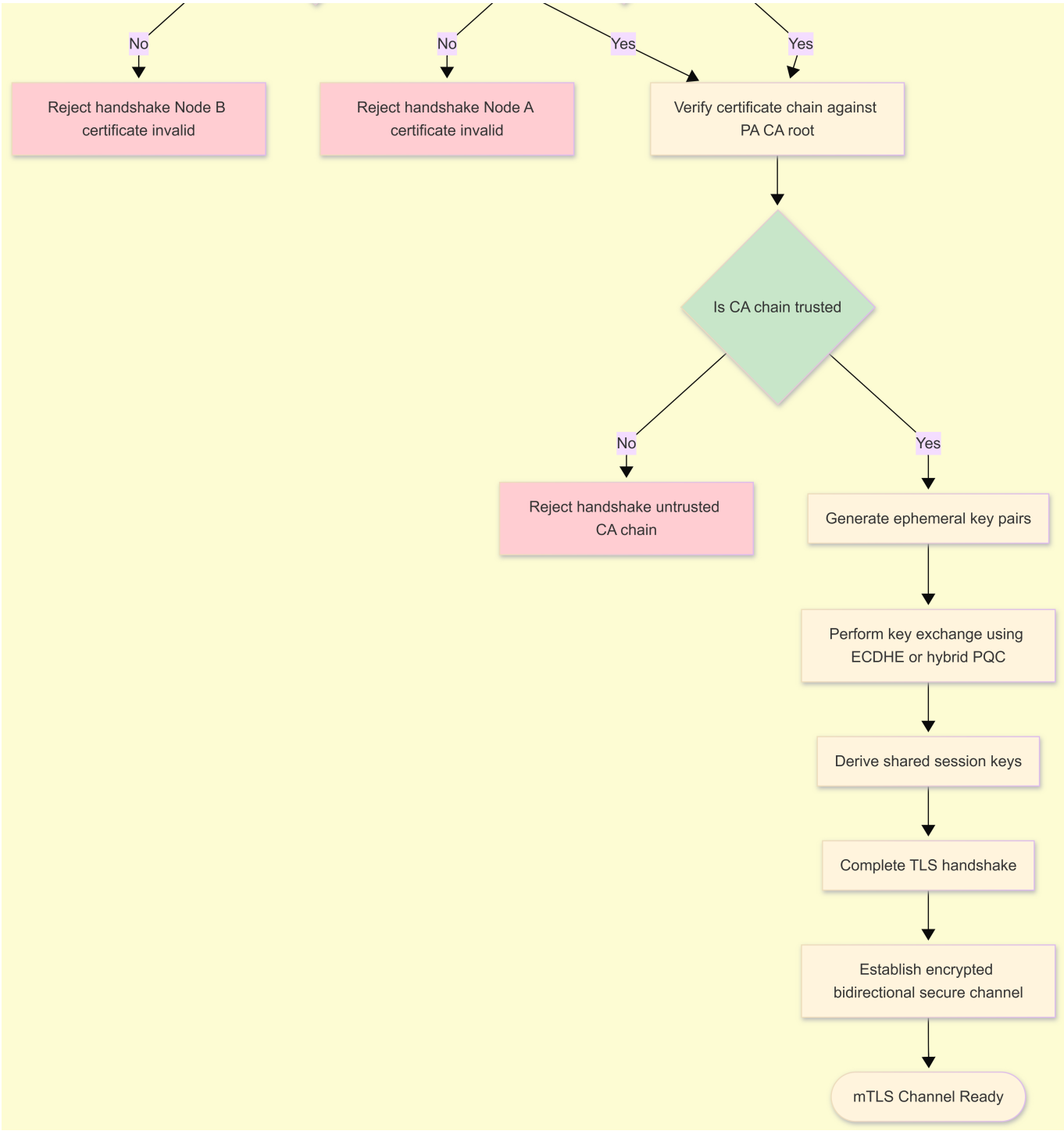
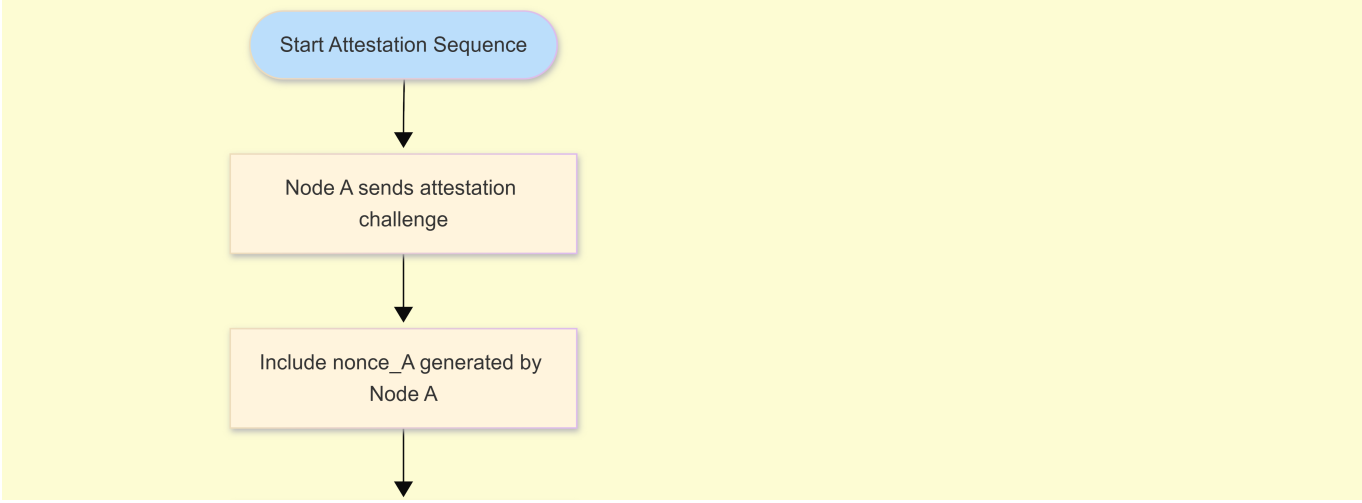
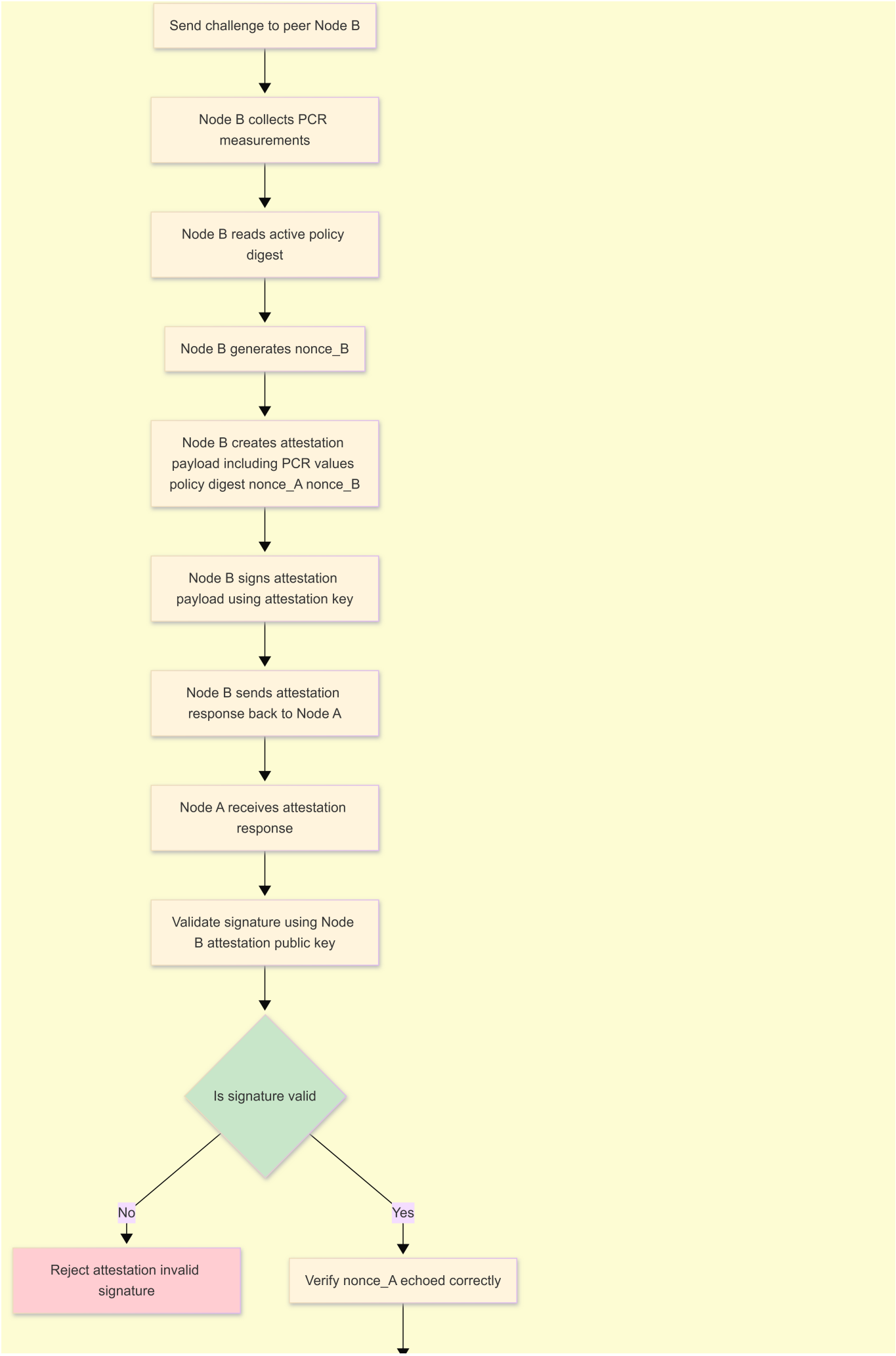
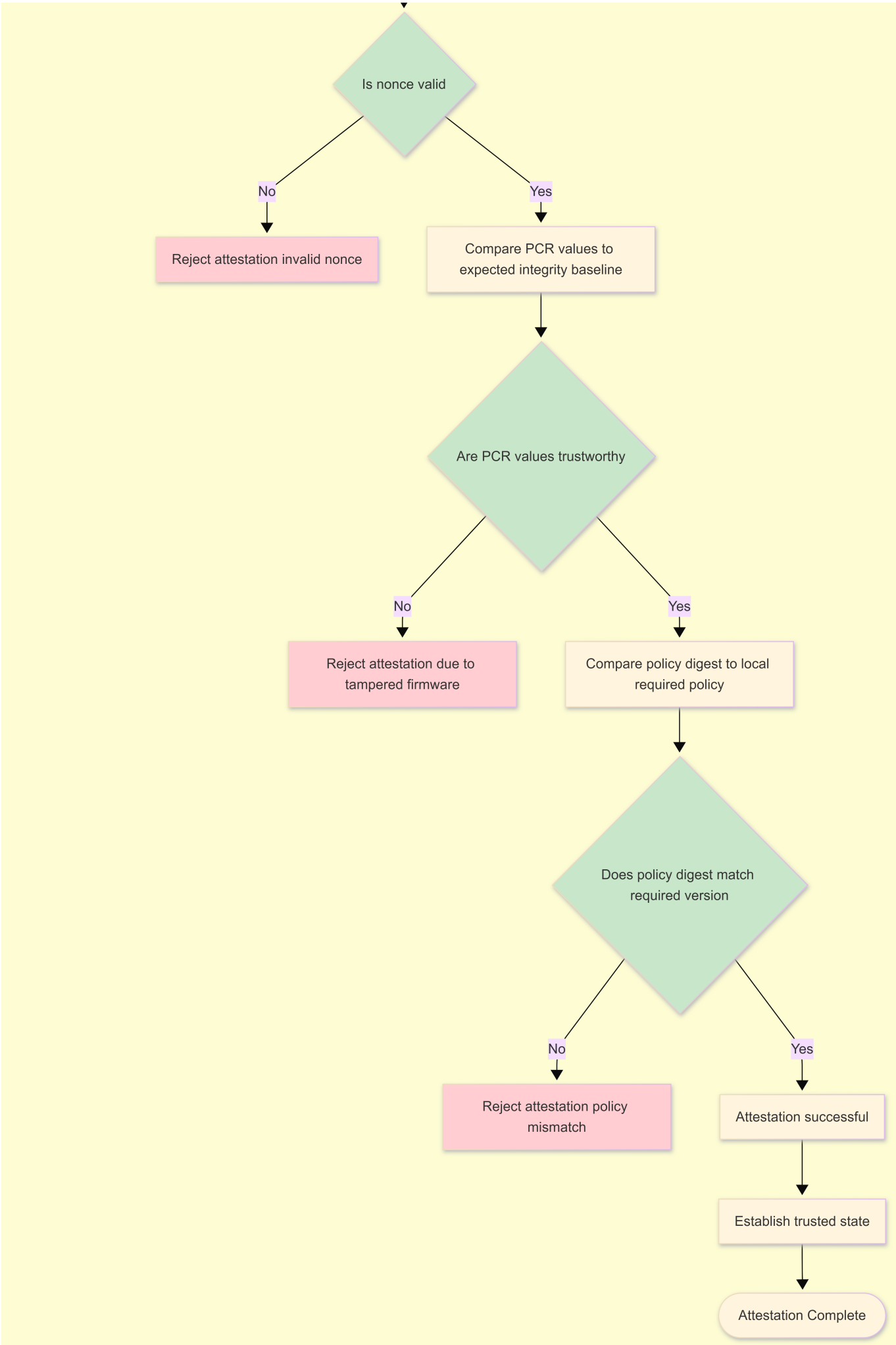


Diagram B-3 — Mutual Attestation




Send challenge to peer Node B
Node B collects PCR measurements
Node B reads active policy digest
Node B generates nonce_B
Node B creates attestation payload including PCR values policy digest nonce_A nonce_B
Node B signs attestation payload using attestation key
Node B sends attestation response back to Node A
Node A receives attestation response
Validate signature using Node B attestation public key
Is signature valid
No
Yes
Reject attestation invalid signature
Verify nonce_A echoed correctly


Is nonce valid
No
Yes
Reject attestation invalid nonce
Compare PCR values to expected integrity baseline
Are PCR values trustworthy
No
Yes
Reject attestation due to tampered firmware
Compare policy digest to local required policy
Does policy digest match required version
No
Yes
Reject attestation policy mismatch
Attestation successful
Establish trusted state
Attestation Complete

2.3.2 Authorization

Once authenticated, how is NodeB authorized?

Answer:

Authorization is policy-based, not certificate-based alone.

NodeA evaluates:

- Role definitions (sensor, controller, gateway, etc.)
- Allowed device IDs / common names
- Policy-defined access rules
- Real-time CRL checks

Policies are:

- Stored locally on each device
- Signed by PA-CA
- Updated atomically (A/B mechanism)

How are authorization changes propagated?

Answer:

- Via epoch-based CRL gossip

-Via policy updates signed by PA-CA

- Devices always apply “highest epoch wins” model
- Revoked or compromised devices are blocked immediately upon receiving CRL

Mini Diagram — Policy-Based Authorization

2.3.3 Secure Channel Establishment

How is the secure session established?

Answer:

The secure channel forms in two stages:

1. Transport Encryption — via QUIC/TLS1.3 (Diagram B-7)

- Provides encrypted tunnel
- Uses ECDHE or PQC-hybrid
- Achieves Perfect Forward Secrecy

2. Trust Binding — via Attestation (Diagram B-3)

- Binds the session to TPM measurements
- policy_digest ensures correct policy execution
- VirtualID binds identity to current software + policy state

Replay attacks are prevented using:

- TLS transcripts
- Nonces
- Sequence numbers
- Session key regeneration
- VirtualID rotation when policy or state changes

If a long-term key is ever compromised, past traffic remains protected due to PFS.

Section 3

3.1 Trust Topology

What is the intended topology?

Answer:
The SG-X network uses a **peer-to-peer full-mesh topology**, where any node that successfully authenticates, attests, and matches policy_digest may securely communicate with any other node.
There is **no runtime hub**, though Cylenium Cloud or a Circle Owner signs policies and CRLs.

Full mesh (every node can trust and talk to every other)?

Answer:
Yes. After mTLS + mutual attestation + policy parity, nodes operate in a **full-mesh Circle of Trust**.

Hub-and-spoke (central controller/gateway, e.g., nodeA is a "trust anchor" or controller)?

Answer:
No. SG-X does not rely on a runtime gateway.
The **PA-CA root** is the trust anchor, not any individual device.

Hierarchical (clusters of nodes with local leaders)?

Answer:
Supported only via **policy-level segmentation** (zones, rooms, roles).
Cryptographic trust remains peer-to-peer.

What is the definition of a "circle of trust" in this context?

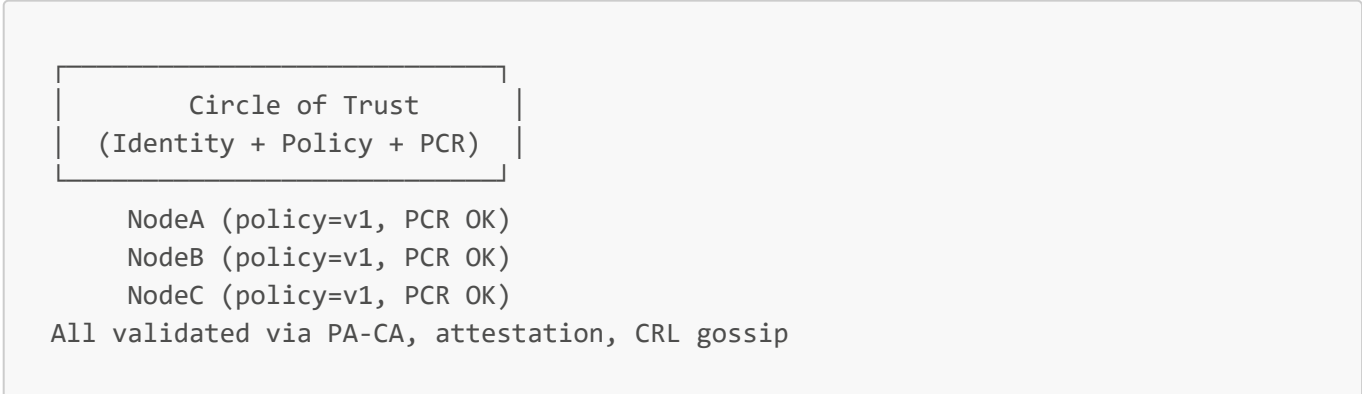
Answer:
A Circle of Trust is defined by:

1. Shared trust anchor (PA-CA or Circle-CA)
2. Matching **policy_digest**
3. Successful **mutual attestation** (PCR + VirtualID)
4. Not revoked (CRL gossip)

Thus, a CoT is:

"Devices enforcing the same signed policy, running trusted firmware, and holding valid, non-revoked identities."

Mini Diagram — Circle of Trust



3.2 Onboarding New Nodes (nodeC, nodeD…)

How is a new node (nodeC) onboarded into the trust domain?

Answer:
Node onboarding includes the following steps:

1. **Discovery** (mDNS or Nebula)
2. **mTLS handshake** (certificate chain → PA-CA)
3. **Mutual attestation** (PCRs + policy_digest)
4. **PolicyParityCheck**
    - If NodeC has outdated policy → receives **PolicyOffer**
5. **VirtualID computation**
6. NodeC joins the Circle of Trust

Out-of-band (OOB) confirmation by an admin? (QR code, PIN, NFC, mobile app)

Answer:
Supported for high-security deployments or human-required approval flows.

Automatic join if it has a valid manufacturer cert + is on the same LAN?

Answer:
Yes. If policy allows auto-admission and attestation succeeds, NodeC may join automatically otherwise it needs to pass the COT verification process.

Who authorizes nodeC to join the circle (human, controller node, cloud)?

Answer:
Authority comes from **PA-signed policy**, which may designate:

- Human admin
- Cloud controller
- Circle Owner
- A designated admitting SG-X device

Does nodeC need mutual attestation with at least one existing member (e.g., nodeA)?

Answer:
Yes. At least one **successful mutual attestation** exchange is required before a node joins the Circle.

Are there approval workflows (“admin must approve requests from new devices”)?

Answer:
Yes. When enabled, onboarding requires:

- Admin approval
- Policy update signed by PA

- Distribution to nodes
- NodeC acceptance only after attestation under new policy_digest

---

Mini Diagram — Onboarding Flow

```
NodeC → Discovery
      → mTLS Handshake
      → Mutual Attestation
      → PolicyParityCheck
           • If outdated → PolicyOffer(v')
      → VirtualID Creation
      → Circle Membership Granted
```

3.3 Scaling Trust & Key Management

How is trust managed when there are many nodes?

Answer:
SG-X scales naturally because trust evaluation is dynamic and distributed.
Nodes rely on:

- Certificate validation
- Attestation (PCR + policy_digest)
- VirtualID
- CRL gossip

No static peer lists, no central directories → supports millions of nodes.

Does each node maintain a list of all peers' certificates?

Answer:
No. Certificates are validated during mTLS.
Nodes do not store large peer lists.

Is there a central directory / endorsement list?

Answer:
Optional via Cloud, but not required for runtime trust.

Are there group keys for:

- Broadcast/multicast messages?
- Group commands ("all nodes in room X, do Y")?

Answer:
SG-X **avoids static group keys** (high security risk).
Instead:

- Each peer uses mTLS
- Group operations follow **policy role-based rules**
- Nebula handles encrypted broadcast/multicast with ephemeral contexts

---

How do you handle key updates when a new node is added?

Answer:
No global rotation required.
NodeC independently completes onboarding.
Other devices remain unaffected.

How do you handle key updates when a node is removed or compromised?

Answer:
SG-X uses **CRL epoch updates**:

1. PA signs new CRL (revoked_spki[])
2. Distributed via **CRL gossip**
3. All nodes instantly fail-closed for revoked identities

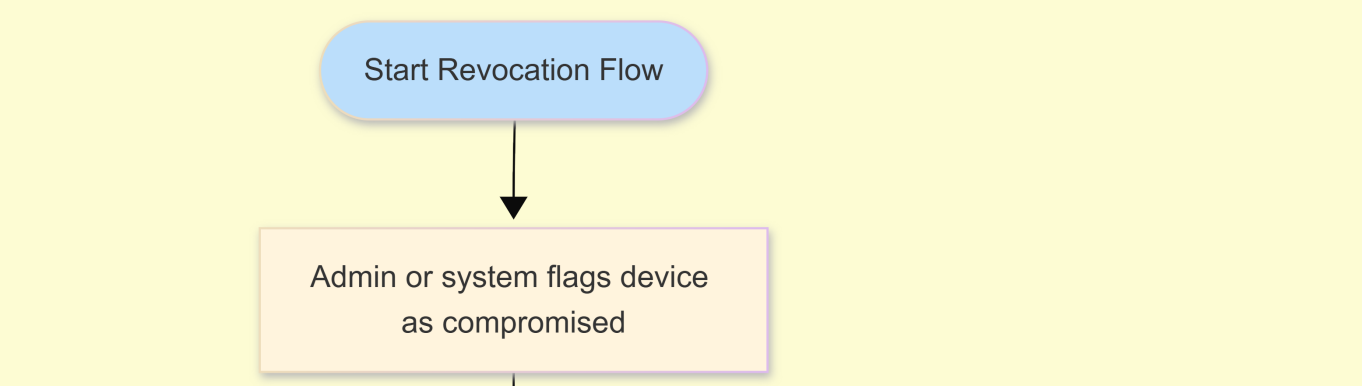
How do you handle key updates when rotating a root CA or intermediate CA?

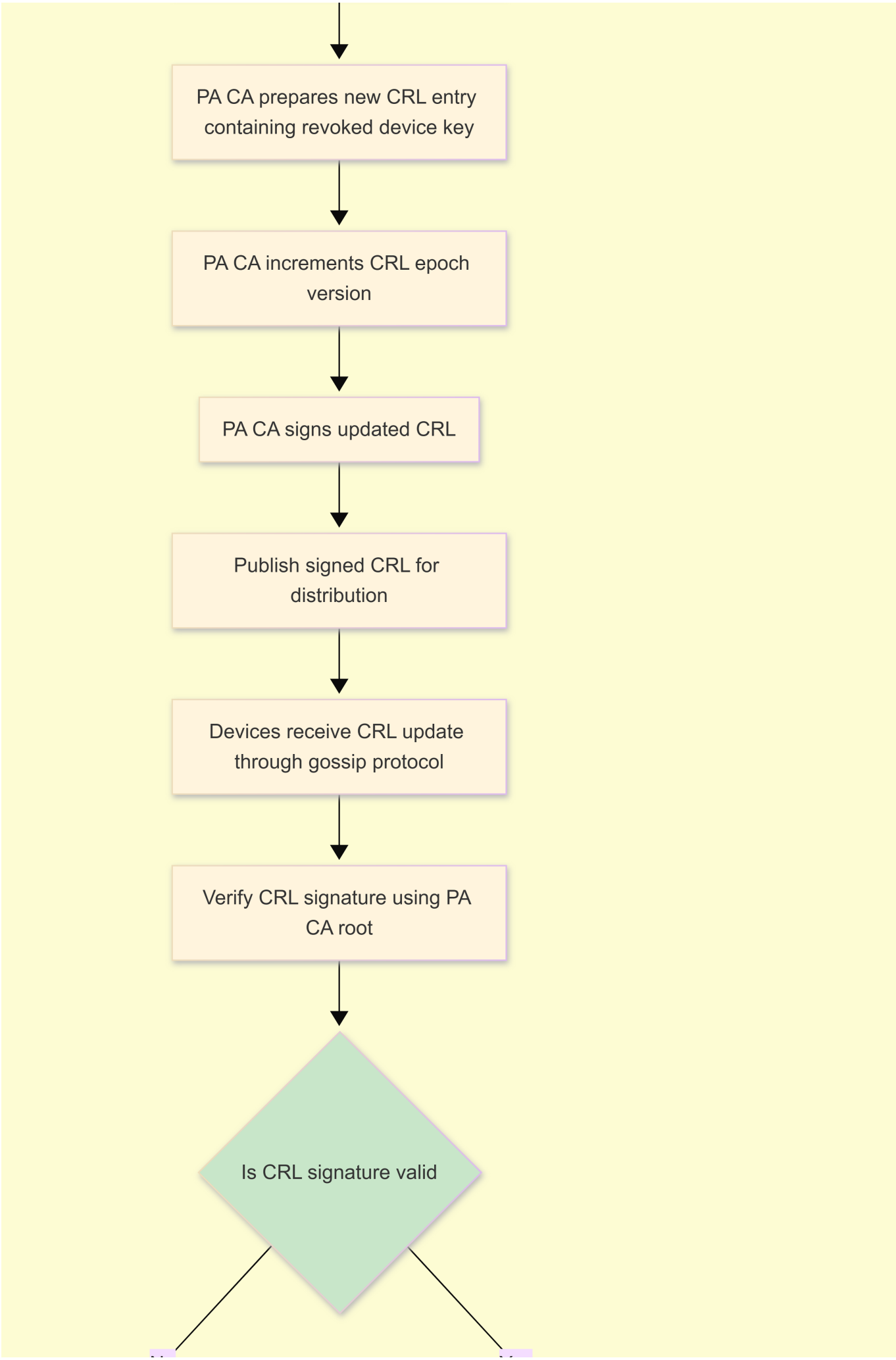
Answer:
Root / intermediate CA rotation proceeds via:

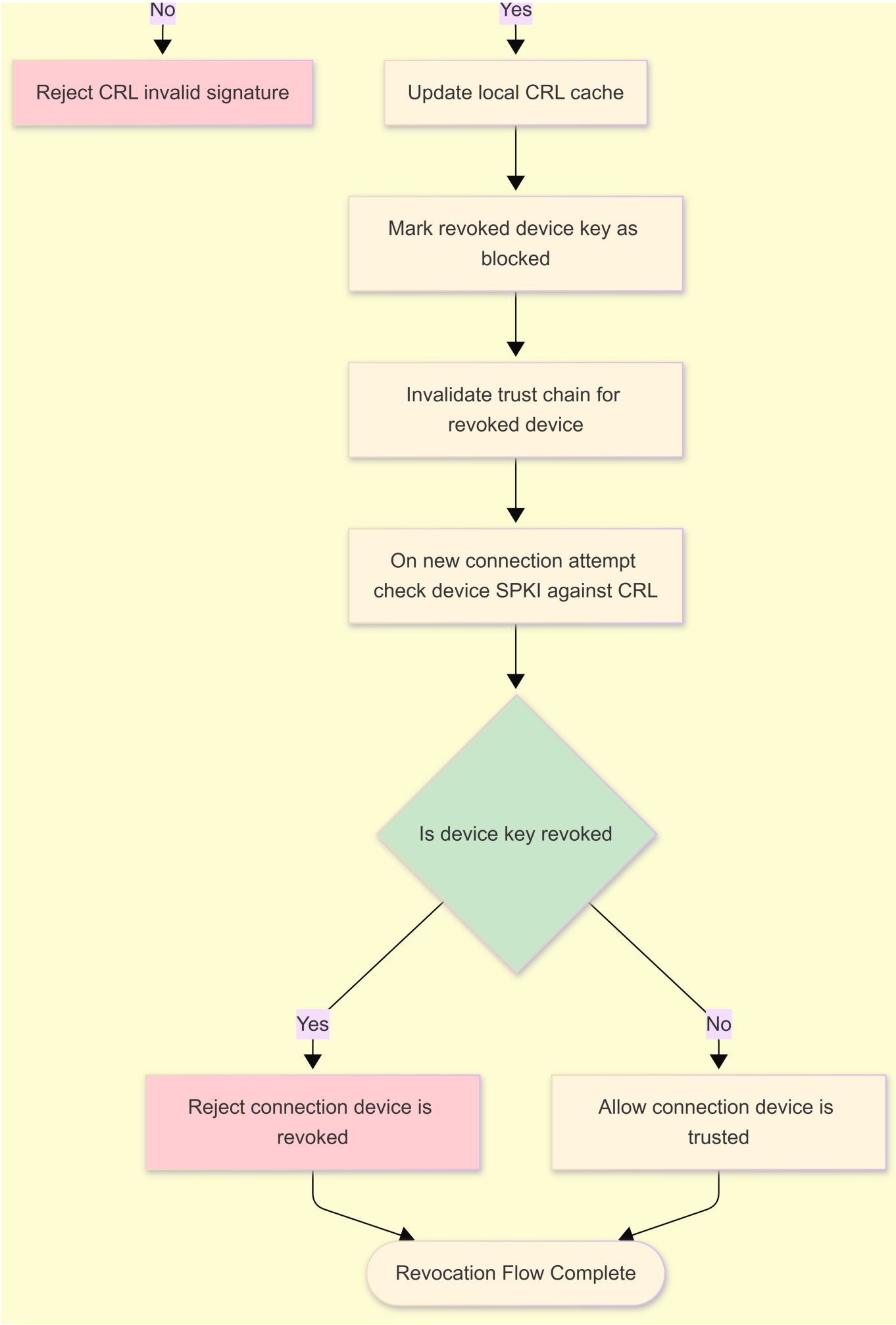
- PolicyOffer including new CA roots
- Atomic A/B install
- PolicyDigest update
- VirtualID rotation

Old CA becomes invalid after activation.

Diagram B-6 — Revocation Gossip




PA CA prepares new CRL entry containing revoked device key
PA CA increments CRL epoch version
PA CA signs updated CRL
Publish signed CRL for distribution
Devices receive CRL update through gossip protocol
Verify CRL signature using PA CA root
Is CRL signature valid


No
Yes
Reject CRL invalid signature
Update local CRL cache
Mark revoked device key as blocked
Invalidate trust chain for revoked device
On new connection attempt check device SPKI against CRL
Is device key revoked
Yes
No
Reject connection device is revoked
Allow connection device is trusted
Revocation Flow Complete

3.4 Roles, Capabilities, and Policy

Are different nodes assigned roles (e.g., sensor, actuator, gateway, admin)?

Answer:
Yes. Policies define device roles:

- Sensor
- Actuator
- Gateway
- Controller
- Admin / Owner

Roles determine allowed behavior and communication permissions.

How does this affect which nodes they trust and what they can do?

Answer:
Cryptographic trust is equal once a node joins the Circle.
However, **policy restricts capabilities**:

- Sensors cannot issue commands
- Controllers can command sensors but cannot alter policy
- Admins/Owners manage onboarding and updates
- Roles strictly regulate command authority

How is policy expressed?

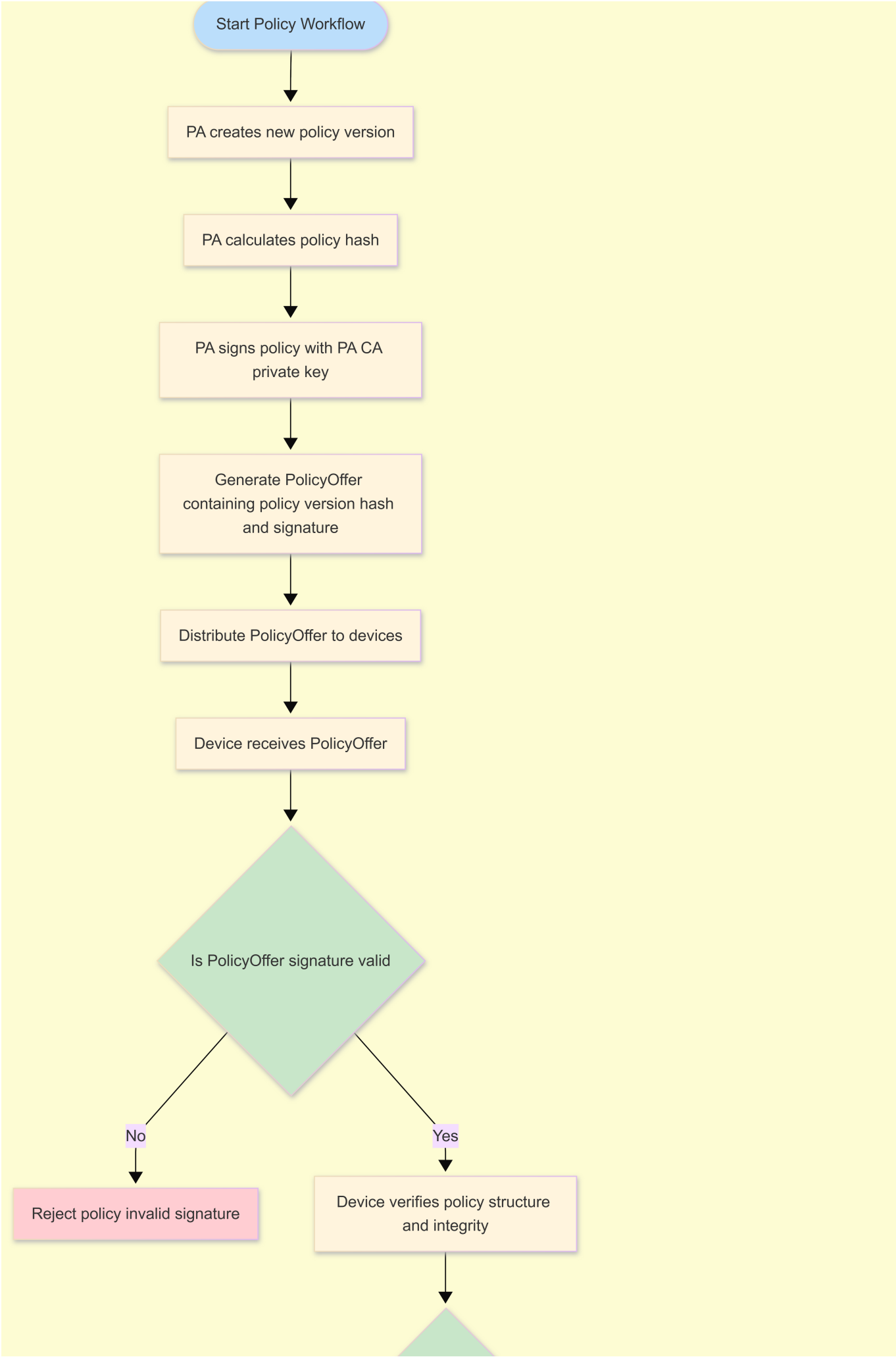
Answer:
Policies are:

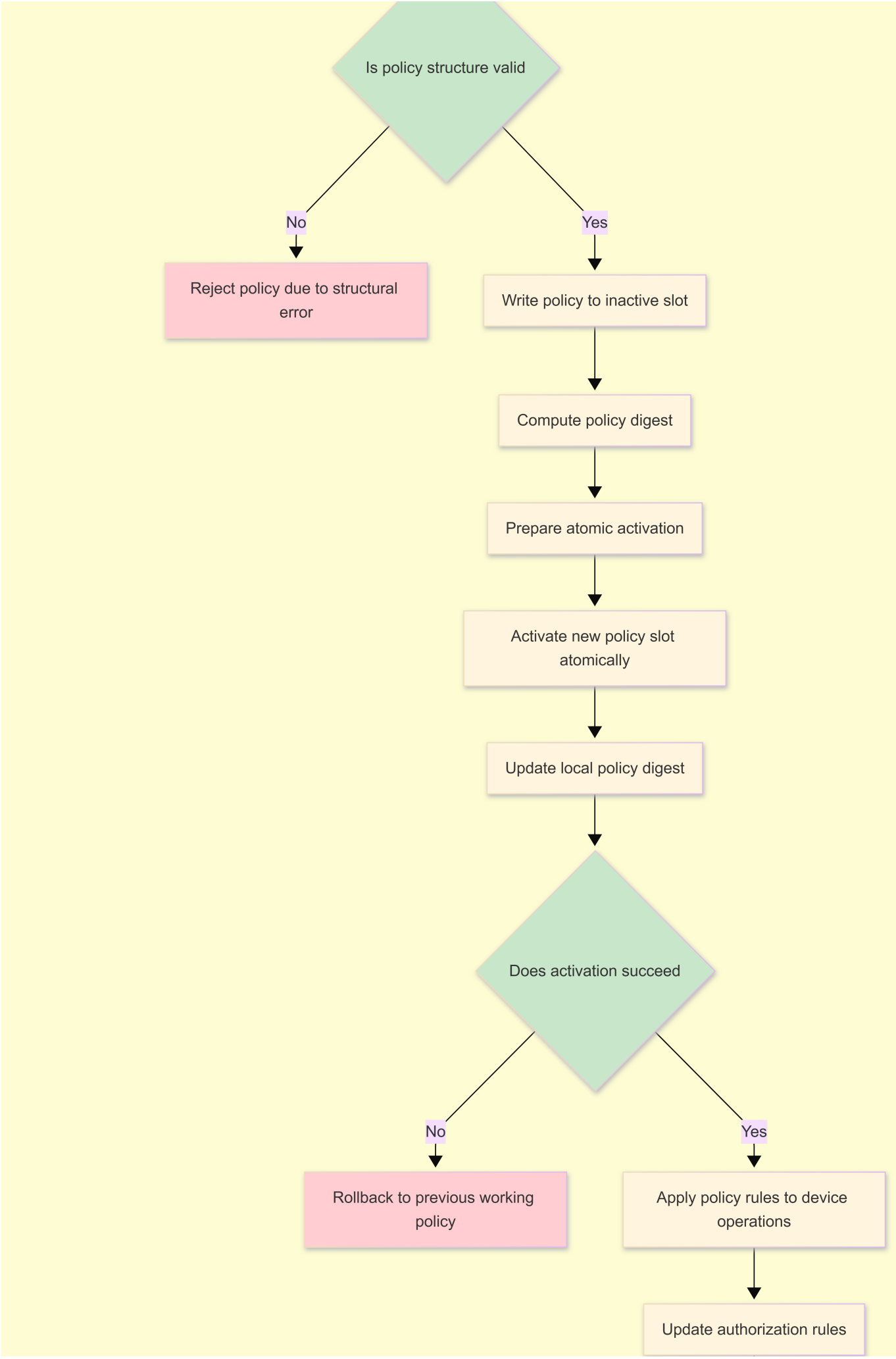
- Canonical JSON
- Signed by PA
- Verified on-device
- Installed atomically using A/B slots
- Hashed into **policy_digest**
- Enforced locally on each node

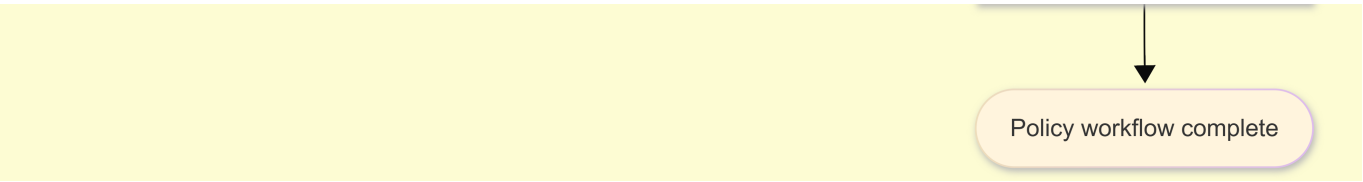
Policies define:

- Allow/deny lists
- Role-based permissions
- Quorum rules
- Data and command restrictions
- Required firmware/model versions

Diagram B-4 — Policy Offer + Atomic Install


Start Policy Workflow
PA creates new policy version
PA calculates policy hash
PA signs policy with PA CA private key
Generate PolicyOffer containing policy version hash and signature
Distribute PolicyOffer to devices
Device receives PolicyOffer
Is PolicyOffer signature valid
No
Yes
Reject policy invalid signature
Device verifies policy structure and integrity


Is policy structure valid
No
Yes
Reject policy due to structural error
Write policy to inactive slot
Compute policy digest
Prepare atomic activation
Activate new policy slot atomically
Update local policy digest
Does activation succeed
No
Yes
Rollback to previous working policy
Apply policy rules to device operations
Update authorization rules



Section 4

4.1 Device Removal / Decommissioning

How is a node intentionally removed from the circle of trust?

Answer:
A node is removed from the Circle of Trust by adding its **device public key (SPKI)** to the **revoked_spki[]** list in a new **CRL epoch**, signed by the PA.
Once revoked, all SG-X devices will refuse connections with that node during the next handshake.

Does an admin "revoke" its certificate?

Answer:
Yes. Certificate revocation is handled through:

- Admin action → Cloud or PA
- PA signs a new **CRL epoch**
- CRL is distributed via **CRL Gossip** (Diagram B-6)

This is the authoritative mechanism for removal.

Are its device keys wiped?

Answer:
If the device is physically accessible during decommissioning:

- A secure wipe command may erase:
  - Identity keys
  - Attestation keys
  - Policies
  - Local state

If the device is lost or inaccessible → **revocation alone** ensures it cannot rejoin.

How quickly do other nodes learn that nodeX is no longer trusted?

Answer:
Nodes learn about the removal **very quickly** through:

1. **CRL Gossip (near-immediate propagation)**
2. **Attestation handshake checks**

3. **Session teardown** when encountering revoked identity

In typical SG-X deployments, full CRL propagation occurs within an expected window of approximately 1–5 minutes for a Circle of around 100 nodes. This timing reflects normal gossip distribution patterns across mixed-network environments. Individual nodes may react sooner during handshake validation, but the complete fleet-wide update follows this window.

Immediate broadcast?

Answer:
Revocation is not a broadcast UDP message; instead, it uses **signed CRL Gossip** to ensure authenticity and ordering.

Next policy sync?

Answer:
Policy sync is **not required** for revocation.
Revocation always occurs through **CRL update**, which is lightweight and fast.

Mini Diagram — Device Removal Workflow

```
Admin → Revoke NodeX
        │
        ▼
PA signs CRL(epoch++, revoked_spki[])
        │
        ▼
CRL Gossip across all SG-X nodes
        │
        ▼
Nodes fail-closed on NodeX during next handshake
```

4.2 Compromised Devices

How is a compromised device detected or signaled?

Answer:
Compromise may be detected through:

1. **Manual admin action**
    - A user reports suspicious behavior
    - Device flagged in Cloud dashboard
2. **UEP / AI-based Intrusion Detection**

- Behavioral anomalies (unexpected commands, traffic patterns)
- Violation of policy constraints
- Virtual Shift alerts

3. **Attestation failure**

- PCR mismatch
- policy_digest mismatch
- Firmware tampering
- Malicious modification of runtime state

Any of these triggers a **revocation workflow**.

Once compromised: Which keys must be considered exposed?

Answer:
If the device is compromised, consider the following keys exposed:

- **Identity Key (public reference leaked)**
  Private key is TPM-protected, but attacker may misuse device while running.
- **Session keys (in memory)**
  Only for active session — thanks to forward secrecy.
- **Cached metadata or policy state**

Keys **NOT** considered exposed:

- Other nodes' keys
- Root CA keys
- Attestation keys of other devices
- Group keys (because SG-X does not use persistent group keys)

How are group keys re-keyed so the compromised node no longer decrypts traffic?

Answer:
SG-X avoids long-lived group keys entirely.

Instead:

- Every connection is **individual mTLS**
- Broadcast/multicast uses **ephemeral Nebula encryption contexts**
- Revocation immediately prevents the compromised node from:
  - Performing mTLS
  - Deriving ephemeral broadcast keys
  - Participating in attested groups

Thus, re-keying is **automatic** and **distributed**.

Are there forensics / logging mechanisms to see what a compromised node did while trusted?

Answer:
Yes. SG-X maintains:

- **Structured logs** (locally and optionally streamed)
- **Command execution logs**
- **Peer identity logs**
- **Attestation history**
- **Policy version history**

These logs allow admins to understand:

- Which nodes NodeX communicated with
- Which commands it issued
- Which data flows occurred while it was trusted

Are there mechanisms to audit which nodes exchanged what kind of messages?

Answer:
Yes.
SG-X supports:

- Metadata-based audit logs
- Connection establishment logs (mTLS identity)
- Policy enforcement logs
- Optional export to SIEM or Cloud
- Optional “Forensic Mode” triggered via policy

Section 5

5.1 Device Identity & Naming

How are devices presented to humans?

Answer:
Devices are presented using a combination of:

- **Serial Number** (manufacturer-defined unique ID)
- **Friendly Name** (user-assigned label, e.g., “Main Door Sensor”)
- **Logical Location** (if defined by policy or admin UI)

The user-facing identity is purposefully distinct from the cryptographic identity (device.crt), which remains internal.

By serial number? Friendly name? Location?

Answer:
All are supported:

- **Serial Number:** Used for shipping, asset tracking, hardware inventory
- **Friendly Name:** Assigned by admins or through mobile app (UX convenience)
- **Location:** Optional metadata set by admin or inferred by policy grouping

These identifiers help humans easily manage devices, without relying on cryptographic details.

How do users/admins verify they are approving the right node during onboarding?

Answer:
Verification uses multiple signals:

1. **Out-of-band metadata**:
    - Serial number
    - QR code encoded with device public key fingerprint
    - NFC tag (optional)
2. **On-screen/mobile app display**:
    - Friendly name suggestion
    - Public key fingerprint
    - Attestation status
    - Manufacturer info (model, version)
3. **Policy-backed confirmation**:
   Admins compare the device's displayed identity with printed identifiers or expected inventory entries.

Are identifiers printed on the device? Displayed in UI?

Answer:
Yes. SG-X supports both:

- **Printed identifiers** on hardware:
    - Serial number
    - QR code containing SPKI fingerprint or device metadata
    - Model/version
- **UI-visible identifiers**:
    - During onboarding flow (mobile app)
    - During approval workflow
    - In admin dashboard or logs

The combination ensures strong UX validation + security correctness.

5.2 Configuration and Updates

How are firmware and configuration updates delivered?

Answer:
Updates are delivered using secure channels:

- **Over-the-air (OTA)** via encrypted Nebula/QUIC/TLS1.3 tunnel
- **Local update** via secure USB or admin tool (optional)

All update bundles are:

- **Digitally signed** by Cylenium / PA
- Versioned and integrity-protected
- Verified in secure boot before activation

Configuration updates (policy changes, parameters, roles) use the **PolicyOffer** and **A/B atomic install mechanism**.

Do they require authenticated / signed updates?

Answer:
Yes — strictly required.

- Firmware images must be **signed** by trusted PA keys
- Policies must be **PA-signed JSON**
- No unsigned or tampered update can pass secure boot validation
- Update chain-of-trust prevents unauthorized firmware or configuration

---

Do they alter trust relationships (e.g., new root CA installed)?

Answer:
Yes — certain updates can alter trust:

1. **Root CA rotation**
   New CA certificates are delivered through **PolicyOffer**, staged in inactive slot, then atomically activated.

2. **Policy updates**
   Updating policy may:

   - Change role permissions
   - Modify onboarding workflows
   - Adjust Virtual Shift quorum rules
   - Add/remove supported firmware versions

3. **Firmware updates**
   May change PCR measurements, resulting in a **new attestation baseline** and **VirtualID rotation**.

All trust-impacting updates follow:

- Signature validation
- Attestation parity
- Atomic activation
- Rollback safety

Is there a rollback strategy if an update breaks trust or protocol compatibility?

Answer:
Yes — SG-X uses an **A/B dual-slot system** for firmware and policy.

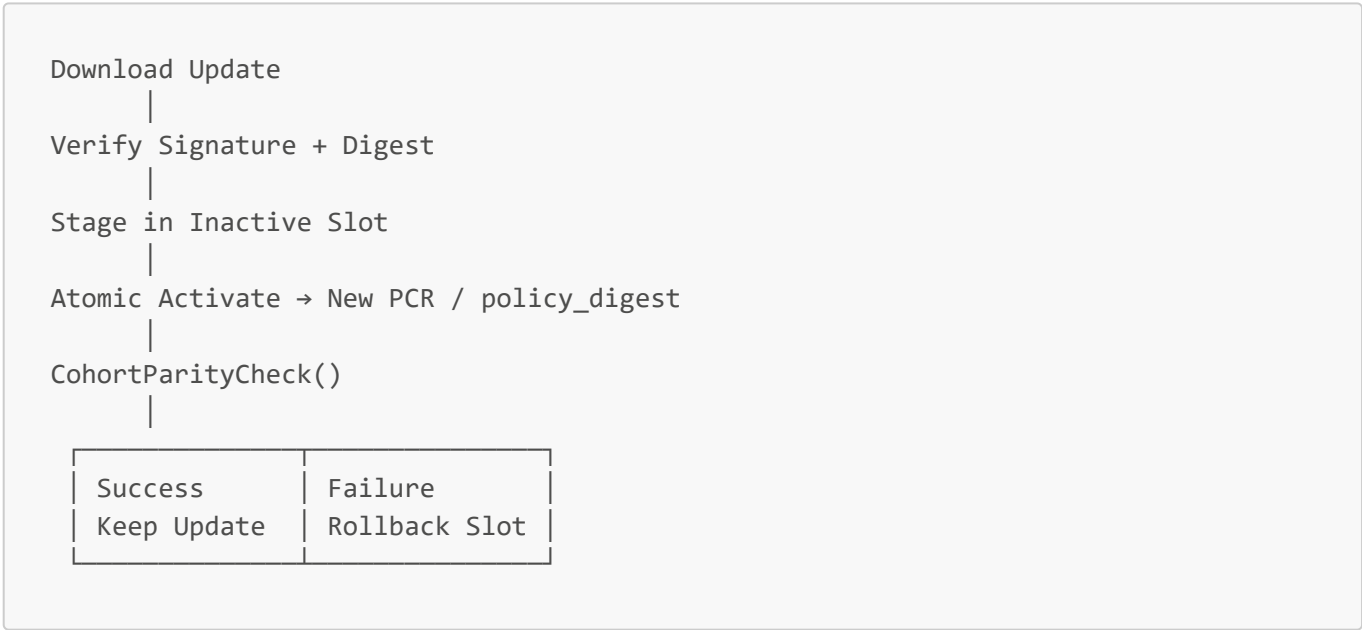
Rollback mechanism:

1. Update is downloaded into **inactive slot**
2. Signature + digest verified
3. Activated atomically
4. **CohortParityCheck** validates system-wide state
5. If failure detected (attestation mismatch, incompatibility):
   → Device automatically reverts to previous **working slot**

Rollback protects against:

- Broken firmware
- Policy mistakes
- CA rotation issues
- Protocol incompatibility

Mini Diagram — Update Workflow (Firmware/Policy)



SECTION 6

Threat Model & Testing

What threat models are explicitly considered?

Answer:
SG-X considers a wide range of realistic and high-capability adversaries. Threat scenarios include:

1. LAN Attacker

- Attacker controls local Wi-Fi or Ethernet
- Can intercept / modify / replay packets
- Mitigated by: QUIC/TLS1.3, ECH, GREASE, mTLS, attestation

2. Rogue Device Attacker

- Fake or malicious device attempts to join
- Fails due to: certificate chain verification, attestation, VirtualID checks

3. Stolen or Lost Device

- Device may be temporarily stolen
- Attacker may attempt physical probing, firmware extraction
- Mitigations: TPM/TEE keys (non-exportable), CRL revocation, secure boot, attestation mismatch

4. Malicious Insider

- A legitimate device trying to exceed permissions
- Mitigated by: role-based policy enforcement, policy_digest validation, command authorization

5. Supply Chain Manipulation

- Attempts to tamper firmware before deployment
- Mitigated by: secure boot, signed firmware images, attestation PCR checks

6. Remote Adversary Over the Internet

- Attacker with full network visibility
- Nebula + QUIC/TLS secure overlay prevents impersonation or leakage

7. Replay or MITM Attacks

- Protected via:
  - TLS transcripts
  - Session keys
  - Nonces
  - VirtualID bound to current trusted state

8. Post-Quantum Threat (Harvest Now, Decrypt Later)

- Hybrid PQC key exchange (X25519 + Kyber) prevents future decryption of intercepted traffic.

What tests validate the Circle of Trust?

SG-X includes structured testing to ensure the Circle of Trust (CoT) behaves as designed.

1. Fake SG-X Device Simulation

Test:
Attempt to join using:

- Self-signed certificate
- Invalid CA chain
- Wrong policy
- Invalid attestation signature

Expected result: **Immediate rejection** during:

- Certificate chain validation
- Mutual attestation
- VirtualID mismatch

---

2. CRL / Revocation Validation

Test:
Revoke a device (NodeX) via new CRL epoch.
Attempt reconnection.

Expected result:

- NodeX rejected on next handshake
- All nodes fail-closed
- No communication permitted
- NodeX cannot rejoin without explicit **re-onboarding + new certificate**

---

3. Policy Parity Testing

Test:
Node attempts communication using outdated or modified policy.

Expected result:

- PolicyParityCheck fails
- PolicyOffer issued
- If device rejects policy or mismatch persists → **connection aborted**

---

4. Attestation Integrity Test

Test:
Tamper PCR values or modify firmware.

Expected result:

- Attestation signature mismatch
- PCR mismatch

- policy_digest mismatch
- Connection denied

---

5. Mutual TLS Enforcement Test

Test:
Attempt to downgrade to one-sided TLS, unsupported cipher suites, or remove certificate.

Expected result:

- Strict mTLS enforced
- Handshake failure
- No downgrade allowed

---

6. Virtual Shift / Anomaly Response Test

Test:
Trigger anomaly detection on one node.
Verify quorum behavior.

Expected result:

- No single node can trigger global shift
- Only M-of-N quorum results in posture change
- Logs capture event for audit

Can you simulate a fake SG-X with self-signed cert and ensure it's rejected?

Answer:
Yes.
The onboarding flow explicitly rejects:

- Self-signed certificates
- Incorrect CA roots
- Invalid certificate chains
- Certificates missing required OIDs
- Certificates failing signature validation

Rejection occurs **before** attestation or policy checks.

Can you confirm that a revoked device can’t rejoin without explicit re-onboarding?

Answer:
Yes.
The CRL mechanism (Diagram B-6) ensures that:

- Revoked device → added to revoked_spki[]
- New CRL epoch signed by PA
- Gossip distributes CRL
- All nodes immediately reject the device on handshake
- Device cannot rejoin unless:
    - Issued a new certificate
    - Re-onboarded as a new identity
    - Passes fresh attestation
    - Matches policy_digest

Are you doing:

Penetration testing?

Answer:
Yes. Planned penetration tests include:

- Network-layer attacks (MITM, replay, downgrade attempts)
- Attestation bypass attempts
- Policy manipulation tests
- Certificate and CRL tampering attempts

Cryptographic protocol review?

Answer:
Yes.
Protocols undergo internal and external review, including:

- QUIC/TLS1.3 implementation
- PQC hybrid KEX path
- mTLS certificate validation
- Attestation signing and verification paths
- CRL epoch logic

Formal verification of the trust logic (if critical enough)?

Answer:
Yes — components with high security impact are subject to:

- Formal modeling of attestation flow
- Finite-state machine verification of policy A/B install
- Formal checking of Circle-of-Trust admission invariants
- Symbolic reasoning for VirtualID uniqueness

Mini Diagram — Threat Model Overview

```
Attacker Models:
 ├─ LAN Attacker
 ├─ Rogue Device
 ├─ Stolen Device
 ├─ Malicious Insider
 ├─ Remote Internet Attacker
 ├─ Firmware Tampering
 └─ Post-Quantum Adversary

SG-X Defenses:
 ├─ mTLS + Attestation
 ├─ Secure Boot + PCR Checks
 ├─ PA-Signed Policies
 ├─ CRL Gossip (Revocation)
 ├─ VirtualID Binding
 └─ PQC Hybrid Encryption
```

Final Recommendation: Attestation Observation Mode

We recommend adding a short Observation Mode before blocking a device after an attestation failure. In the current model, any temporary variance in PCR values, policy_digest state, early boot conditions, or timing delays may cause an immediate rejection—even when the device is actually healthy and simply stabilizing. In field environments, especially during first boot, policy rollout, or firmware updates, these transient states are common and non-malicious.

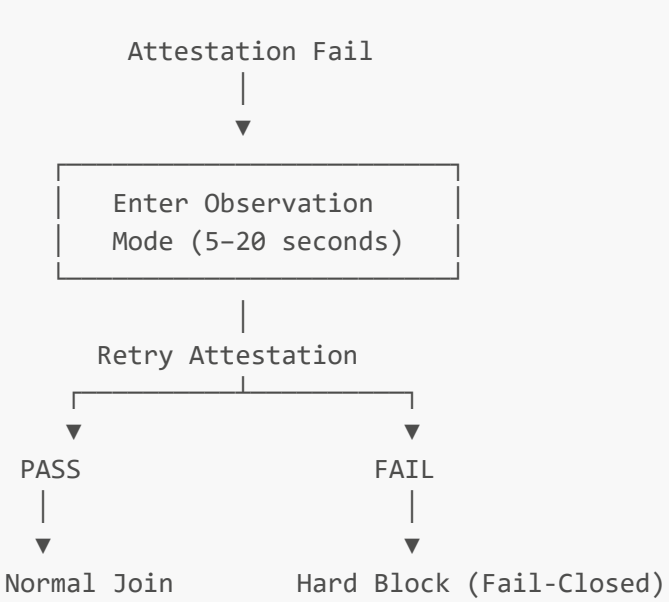
Observation Mode flow:

- Attestation fails → device enters a 5–15 second observation window.
- Device retries attestation once or twice while allowing PCR and policy state to stabilize.
- If the retry succeeds → device joins normally.
- If the mismatch persists → device is blocked as usual (fail-closed).

Benefits:

- Prevents unnecessary lockouts from harmless boot-time instability
- Improves onboarding and update reliability
- Reduces field support issues
- Maintains full zero-trust security (final decision still strictly validated)

Simple Diagram — Attestation Observation Mode



Policy Preview Mode

Current Drawback in SG-X Policy Workflow

In the current SG-X design, when a device receives a new PolicyOffer from PA-CA:

1. The policy is immediately written into the inactive slot
2. The structure and signature are validated
3. The device atomically activates the new policy
4. If activation leads to incorrect behavior, the device performs a rollback

This approach is cryptographically secure, but operationally it creates a short window where a misconfigured or unintended policy may be briefly applied. Even a few seconds of incorrect policy enforcement can result in:

- Temporary denial of expected communications
- Incorrect or unexpected actuator/sensor behavior
- Short-lived instability during onboarding or updates
- Difficult-to-debug intermittent issues, especially in large deployments

Rollback corrects the state, but the temporary inconsistent state still poses operational risk.

Recommended Solution: Policy Preview Mode

We recommend introducing a "Policy Preview Mode" to validate and simulate incoming policies before activation.

When a PolicyOffer is received, the device should:

1. Load the policy into memory without writing it to the inactive slot
2. Verify the signature and schema
3. Perform a local simulation of the policy to check:
    - Role and permission changes
    - Communication and access rules

    - Firmware or version requirements
    - Conflicting or unsafe configurations
4. Proceed with normal A/B installation only if the preview succeeds
5. Reject the policy immediately if preview validation fails, without activating or rolling back any slot

---

Benefits

- Prevents devices from entering temporary unstable states
- Avoids unnecessary rollbacks during policy distribution
- Improves reliability during large-scale or staggered policy rollouts
- Reduces operational and support complexity
- Maintains full zero-trust security, as final activation still requires complete validation
- Ensures that only fully validated policies ever affect runtime behavior